NXB Ledger CTF — Solve Guide

How all six stages are solved, step by step. Six-stage forensic investigation: OSINT → decoding → BOLA → forensics → log analysis → reconstruction. Red/black hacker theme on CTFd.

How to use this guide

The six stages form a single chain. Each stage's answer unlocks the next (stages 2–6 are visible but locked until the previous stage is solved). Always test with a normal player account, not the admin account. On the local VM every service is reached at `127.0.0.1`; if players connect from other machines, replace it with the VM's LAN IP (the `HOST_IP` value in `.env`).

Service	Address	Used in
CTFd (game, submissions, scoreboard)	`http://127.0.0.1:8000`	All stages
Loan portal	`http://127.0.0.1:8080`	Stage 3
Broker classifieds	`http://127.0.0.1:8081`	Stage 1
Evidence checker	`http://127.0.0.1:8082/check/stageN`	Stages 1, 4, 5, 6

The evidence checker accepts a browser form (open the URL) or a JSON POST with `curl`. Both give the same result. Flag values themselves are organizer-only and live in `evidence-generation/output/secrets/flags.json`; enter each flag in CTFd as type **Static**.

> **Before you start:** if the stack is stopped, bring it up with `newgrp docker` (wait for the prompt to return), then `cd ~/ctf/nxb-ledger-ctf && docker compose up -d`. Challenge descriptions must use `127.0.0.1`, not the old `192.168.56.10` address.

STAGE 1 Find the broker ad — OSINT

What you are given

The broker classifieds site on port 8081 carries many loan-broker ads. One of them is the ad that leaked the application you will chase through the whole game.

How to solve it

1. Open the classifieds at `http://127.0.0.1:8081` and read the ads. Do not pick by wording alone: two ads (PUB-013 and PUB-017) are worded similarly on purpose.
2. Match the identifying details instead. The correct ad is **PUB-017**, tied to broker code **BRK17**. Cross-check the post ID and broker code together; that pair is unique even where the sales wording overlaps.
3. From that ad, note the four values the checker asks for: the **post_id**, the **contact_code**, and the two distinguishing **features** of the ad (`feature1`, `feature2`).
4. Submit them to the checker, either in the browser form at `http://127.0.0.1:8082/check/stage1` or with curl:

```
curl -s -X POST http://127.0.0.1:8082/check/stage1 \
  -H 'Content-Type: application/json' \
  --data-raw '{"post_id":"...","contact_code":"...","feature1":"...","feature2":"..."}'
```

> **Answer shape:** post **PUB-017** / broker **BRK17**, with that ad's contact code and two features. A correct POST returns the Stage 1 flag; paste it into CTFd to unlock Stage 2.

Lesson: in OSINT, identifiers beat descriptions. Ads can copy each other's wording; post IDs and broker codes cannot.

STAGE 2 Decode the training message

What you are given

A training message in encoded form, plus a hash manifest so you can prove the file you decoded is the file that was issued.

How to solve it

1. Take the encoded message body and Base64-decode it. On the VM: `base64 -d message.txt > decoded.txt` (or paste the body into any Base64 decoder). Read the decoded text carefully; it carries the Stage 2 instruction and flag material.
2. Verify integrity before trusting it: run `sha256sum` on the downloaded evidence file and compare the output with the value in `manifest_sha256.txt`. The two hashes must match exactly; if they do not, re-download the file rather than decoding a corrupt copy.
3. Submit the decoded result / flag in CTFd to unlock Stage 3.

> **Answer shape:** the Base64-decoded training message, hash-verified against the manifest. Flags in this deployment use the format `NXB{S0n_<32 hex>}`.

Lesson: decode first, verify always. The hash check is what turns "a message I found" into evidence.

STAGE 3 Beyond the Branch — BOLA on the loan portal

What you are given

Access to the loan portal on port 8080. Log in with the training officer account: username `br03_officer`, password `Training#2026`.

How to solve it

1. Log in and open your own loan application. Look at the URL: the application is identified by an ID in the address, in the form `APP-xxxx`.
2. The vulnerability is a Broken Object Level Authorization (BOLA) flaw: the portal checks that you are logged in, but never checks that the application in the URL belongs to you.
3. Change the application ID in the URL to **APP-0842** — the application from the leaked evidence in Stages 1–2 — and load the page. Another customer's loan file opens.
4. Read that file. The Stage 3 flag is inside it. Submit the flag in CTFd to unlock Stage 4.

> **Answer:** the flag inside application **APP-0842**'s file, reached by editing the ID in the URL after logging in as `br03_officer`.

Lesson: "logged in" is not "authorized." Any URL that names an object must check that the object belongs to the caller. APP-0842 now becomes the target application for Stages 4, 5 and 6.

STAGE 4 Verify the leaked screenshot — forensics

What you are given

The leaked screenshot at full resolution, the asset register (`asset_register.csv`), the capture-event log (`capture_events.csv`), and a hash manifest (`manifest_sha256.txt`).

How to solve it

1. **Zoom into the watermark.** The bottom-right corner of the screenshot reads `NXB-BR03-WS07` . That is the workstation the screenshot was taken on. The screenshot also shows "Logged in as: EMP042 (BR03)".
2. **Corroborate with the asset register.** `NXB-BR03-WS07` is listed there: branch BR03, assigned to EMP042, status active. It matches the login shown in the screenshot. (Workstation WS09 is decommissioned — a distractor.)
3. **Verify integrity.** Run `sha256sum` on the downloaded screenshot and compare it with `manifest_sha256.txt` . The hashes must match before you treat the image as evidence.
4. **Find the capture event.** In `capture_events.csv` , find the row matching *both* that workstation *and* the target application APP-0842: `CAP-0312, NXB-BR03-WS07, APP-0842, 2026-08-10T09:18:00+05:30, screen_capture` . The other CAP rows are decoys with different workstations or applications.
5. Submit both values to the checker form at `http://127.0.0.1:8082/check/stage4` , or with curl:

```
curl -s -X POST http://127.0.0.1:8082/check/stage4 \
  -H 'Content-Type: application/json' \
  --data-raw '{"workstation_id":"NXB-BR03-WS07","capture_event":"CAP-0312"}'
```

> **Answer:** workstation_id `NXB-BR03-WS07` , capture_event `CAP-0312` .

Lesson: one artifact is a claim; three agreeing artifacts are evidence. The watermark, the register and the log each say the same thing independently.

STAGE 5 Build the timeline — log analysis

What you are given

An evidence bundle: `auth.csv` , `app_access.csv` , `workstation_sessions.csv` , `leave.csv` , and an alert note (`alert_note.txt`) flagging a failed-login event.

How to solve it

Build the timeline for the target application APP-0842 and test whether the highlighted failed login belongs to it. The real chain, in time order (+05:30):

Time	Event	Record
09:12:00	EMP042 authenticates successfully on NXB-BR03-WS07	AUTH-0002
09:12:05	Session SES-91A opens (EMP042 @ NXB-BR03-WS07)	workstation_sessions.csv
09:16:00	EMP042 accesses APP-0842 inside SES-91A	ACC-0501
09:18:00	Screen capture of APP-0842 on that workstation (Stage 4)	CAP-0312
09:40:00	Session SES-91A closes	workstation_sessions.csv

The alert note points at failed-login event **FAIL-009**. Check it against the chain: it is a failed login for **EMP019** at **08:42:00** from **NXB-BR02-WS03** (confirmed in `auth.csv`). Wrong employee, wrong workstation, and 30 minutes before the real session — and it has no matching successful session or application-access record. It is not part of the causal chain. The `leave.csv` entry showing EMP042 on leave that day is the second distractor: the auth, session and access records prove the *account* was active regardless of the leave roster.

```
curl -s -X POST http://127.0.0.1:8082/check/stage5 \
  -H 'Content-Type: application/json' \
  --data-raw '{"session_id":"SES-91A","employee_id":"EMP042","unrelated_failed_event":"FAIL-009"}'
```

> **Answer:** session_id `SES-91A`, employee_id `EMP042`, unrelated_failed_event `FAIL-009`.

Lesson: the loudest alert is not always the relevant one. The timeline decides, not the alert note.

STAGE 6 Reconstruct the chain — correlation script

What you are given

A larger chat export (`messages.jsonl`, 200+ lines with Base64 bodies), an employee directory (`directory.csv`), a broker ledger (`broker_ledger.csv`), and a schema note (`SCHEMA.md`). You also reuse two Stage 5 files: `app_access.csv` and `workstation_sessions.csv`. The task: write `correlate.py` that produces `evidence.json` with six fields, and be ready to explain at least one join in the viva.

How to solve it — the four joins

1. **Filter the noise.** Only three messages mention the target application APP-0842. CHAT-77: `nxb_ruwan` to ghostbroker17, "app APP-0842 confirmed, payment ref TX-118 incoming." CHAT-78 is ghostbroker17 replying "transfer sent, see ledger." CHAT-76 asks "is the file ready?" with no payment reference.
2. **Apply the rule.** Only a chain linking an *employee* to a *payment record for the application* counts. CHAT-77's sender `nxb_ruwan` maps through `directory.csv` to **EMP042** (an employee), and its reference TX-118 maps through `broker_ledger.csv` to "payment ref linked to APP-0842". That chain wins. CHAT-78's sender maps to `N/A` in the directory (an external contact, not an employee), and CHAT-76 has no transaction reference, so both fail.
3. **Join the session trail.** Take (EMP042, APP-0842) into `app_access.csv` to get session **SES-91A**, then take SES-91A into `workstation_sessions.csv` to get workstation **NXB-BR03-WS07**.
4. **Notice the convergence.** The same employee, application, workstation and session appear in Stages 3, 4, 5 and now 6. That convergence is the reconstruction.

> **Answer (all six fields):** employee_id `EMP042` · application_id `APP-0842` · workstation_id `NXB-BR03-WS07` · session_id `SES-91A` · broker_transaction_ref `TX-118` · message_id `CHAT-77`.

Run it

Put all five evidence files plus the script in one folder — the script reads from the current directory:

```
mkdir -p ~/ctf/stage6 && cd ~/ctf/stage6
# copy in: messages.jsonl directory.csv broker_ledger.csv
#          app_access.csv workstation_sessions.csv   (Stage 5 files)
python3 correlate.py
curl -s -X POST http://127.0.0.1:8082/check/stage6 \
  -H 'Content-Type: application/json' \
  --data-raw @evidence.json
```

The script below derives the answer from the joins instead of hard-coding it, so it still works if the evidence is reshuffled — which is exactly what the viva asks about.

```
#!/usr/bin/env python3
"""Stage 6: reconstruct the evidence chain for the target application."""
import base64, csv, json

TARGET_APP = "APP-0842"  # application under investigation (Stages 3/4/5)

def read_csv(p):
    with open(p, newline="") as f:
        return list(csv.DictReader(f))

def read_jsonl(p):
    with open(p) as f:
        return [json.loads(l) for l in f if l.strip()]

def main():
    directory = {r["handle"]: r for r in read_csv("directory.csv")}
    ledger = {r["tx_ref"]: r for r in read_csv("broker_ledger.csv")}
    access_rows = read_csv("app_access.csv")
    sessions = {r["session_id"]: r for r in read_csv("workstation_sessions.csv")}

    results = []
    for m in read_jsonl("messages.jsonl"):
        if m["application_id"] != TARGET_APP or not m["transaction_ref"]:
            continue
        body = base64.b64decode(m["body_b64"]).decode("utf-8")
        employee_id = directory.get(m["sender_handle"], {}).get("employee_id")
        if not employee_id or employee_id == "N/A":
            continue  # must link an *employee*
        payment = ledger.get(m["transaction_ref"])
        if not payment or TARGET_APP not in payment.get("note", ""):
            continue  # payment must reference the target application
        acc = next((a for a in access_rows
                    if a["employee_id"] == employee_id
                    and a["application_id"] == TARGET_APP), None)
        sess = sessions.get(acc["session_id"]) if acc else None
        if not sess:
            continue
        results.append({
            "employee_id": employee_id,
            "application_id": TARGET_APP,
            "workstation_id": sess["workstation_id"],
            "session_id": acc["session_id"],
            "broker_transaction_ref": m["transaction_ref"],
            "message_id": m["message_id"],
        })

    assert len(results) == 1, f"expected one chain, got {len(results)}"
    with open("evidence.json", "w") as f:
        json.dump(results[0], f, indent=2)
    print(json.dumps(results[0], indent=2))

if __name__ == "__main__":
    main()
```

For the viva, the join to explain is messages → directory: a chat handle means nothing until the directory turns it into an employee ID, and that single step is what eliminates the external broker's messages from the chain.

Quick-reference: every stage at a glance

Stage	Skill	Key move	Submit
1	OSINT	Match PUB-017 / BRK17 by ID, not wording	post_id, contact_code, feature1, feature2 → `/check/stage1`
2	Decoding	Base64-decode; sha256-verify against manifest	Decoded flag → CTFd
3	BOLA	Edit URL ID to APP-0842 after login	Flag inside APP-0842 file → CTFd
4	Forensics	Watermark + register + capture log agree	NXB-BR03-WS07, CAP-0312 → `/check/stage4`
5	Log analysis	Timeline beats the alert note	SES-91A, EMP042, FAIL-009 → `/check/stage5`
6	Reconstruction	Four joins converge on one chain	Six fields from evidence.json → `/check/stage6`

One thread runs through the whole game: broker ad (Stage 1) → decoded message (Stage 2) → application APP-0842 (Stage 3) → workstation NXB-BR03-WS07 and capture CAP-0312 (Stage 4) → session SES-91A and employee EMP042 (Stage 5) → payment TX-118 in message CHAT-77 (Stage 6). If a later stage's answer does not connect back to APP-0842, EMP042, NXB-BR03-WS07 and SES-91A, it is a decoy — go back to the evidence.
